

CCES – College of Computer Engineering and Sciences

CS3401 – Algorithm Design and Analysis

Chapter 6- Transform & Conquer

References:

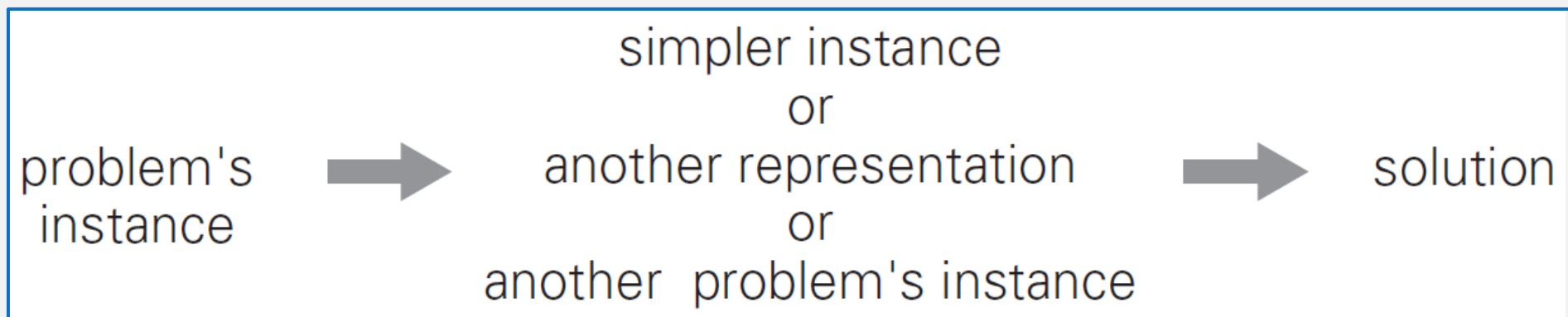
- ❖ *Introduction to the Design and Analysis of Algorithms. Anany V. Levitin, Pearson 3rd Edition.*

Transform & Conquer

Introduction

The ***Transform-and-conquer*** technique is a group of design methods that are based on the idea of transformation.

- ❑ Transformation to a simpler or more convenient instance of the same problem—***instance simplification***.
- ❑ Transformation to a different representation of the same instance—***representation change***.
- ❑ Transformation to an instance of a different problem for which an algorithm is already available—***problem reduction***.



Transform & Conquer

Instance simplification - Presorting

Solve a problem's instance by transforming it into another simpler/easier instance of the same problem.

Presorting

Many problems involving lists are easier when list is sorted, e.g.

- searching
- computing the median (selection problem)
- checking if all elements are distinct (element uniqueness)

Also:

- Topological sorting helps solving some problems for dags.
- Presorting is used in many geometric algorithms.

Transform & Conquer

Instance simplification - Presorting

EXAMPLE 1 *Checking element uniqueness in an array*

- Brute force algorithm
Compare all pairs of elements Efficiency: $O(n^2)$
- Presorting-based algorithm
 - Stage 1: sort by efficient sorting algorithm (e.g. mergesort)
 - Stage 2: scan array to check pairs of adjacent elements
 - Efficiency: $\Theta(n \log n) + O(n) = \Theta(n \log n)$

ALGORITHM *PresortElementUniqueness*($A[0..n - 1]$)
//Solves the element uniqueness problem by sorting the array first
//Input: An array $A[0..n - 1]$ of orderable elements
//Output: Returns “true” if A has no equal elements, “false” otherwise
sort the array A
for $i \leftarrow 0$ **to** $n - 2$ **do**
 if $A[i] = A[i + 1]$ **return false**
return true

Transform & Conquer

Instance simplification - Presorting

EXAMPLE 2 *Computing a mode*

- Definition:
a **mode** is a value that occurs most often in a given list of numbers:
e.g. for 2, 6, 7, 8, 6, 1, 6, 3, 6, 2 the mode is 6
- Brute force algorithm:
 - Check the frequency of occurrence of each element and select element with highest frequency
 - Efficiency: $O(n^2)$
- Presorting-based algorithm:
 - Stage 1: sort by efficient sorting algorithm (e.g. mergesort)
 - Stage 2: scan array counting equal adjacent elements
 - Efficiency: $\Theta(n \log n) + O(n) = \Theta(n \log n)$

Transform & Conquer

Instance simplification - Presorting

EXAMPLE 2 *Computing a mode*

```
ALGORITHM PresortMode( $A[0..n - 1]$ )  
//Computes the mode of an array by sorting it first  
//Input: An array  $A[0..n - 1]$  of orderable elements  
//Output: The array's mode  
sort the array  $A$   
 $i \leftarrow 0$  //current run begins at position  $i$   
 $modefrequency \leftarrow 0$  //highest frequency seen so far  
while  $i \leq n - 1$  do  
     $runlength \leftarrow 1$ ;  $runvalue \leftarrow A[i]$   
    while  $i + runlength \leq n - 1$  and  $A[i + runlength] = runvalue$   
         $runlength \leftarrow runlength + 1$   
    if  $runlength > modefrequency$   
         $modefrequency \leftarrow runlength$ ;  $modevalue \leftarrow runvalue$   
     $i \leftarrow i + runlength$   
return  $modevalue$ 
```

Transform & Conquer

Instance simplification - Presorting

EXAMPLE 3 *Searching problem*

- Problem:

Search for a given k in $A[0..n-1]$

Presorting-based algorithm:

- Stage 1 Sort the array by an efficient sorting algorithm
- Stage 2 Apply binary search
- Efficiency: $\Theta(n \log n) + O(\log n) = \Theta(n \log n)$

Transform & Conquer

Taxonomy of Searching Algorithms

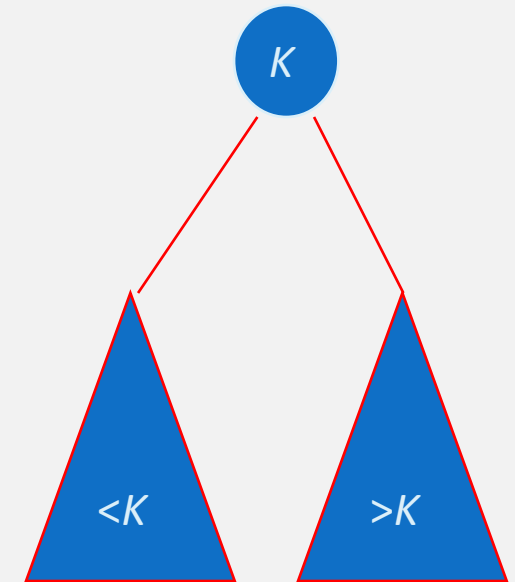
- List searching
 - sequential search
 - binary search
 - interpolation search
- Tree searching
 - binary search tree
 - binary balanced trees: AVL trees, red-black trees
 - multiway balanced trees: 2-3 trees, 2-3-4 trees, B trees
- Hashing
 - open hashing (separate chaining)
 - closed hashing (open addressing)

Transform & Conquer

Binary Search Tree

Dictionary Operations on Binary Search Trees

- Arrange keys in a binary tree with the *binary search tree property*.
- Searching – straightforward
- Insertion – search for key, insert at leaf where search terminated
- Deletion – 3 cases:
 - deleting key at a leaf
 - deleting key at node with single child
 - deleting key at node with two children
- Efficiency depends of the tree's height: $\lfloor \log_2 n \rfloor \leq h \leq n-1$,
with height average (random files) be about $3\log_2 n$
- Thus all three operations have
 - worst case efficiency: $\Theta(n)$
 - average case efficiency: $\Theta(\log n)$
- Bonus: inorder traversal produces sorted list.



Transform & Conquer

Instance simplification - Balanced Search Tree

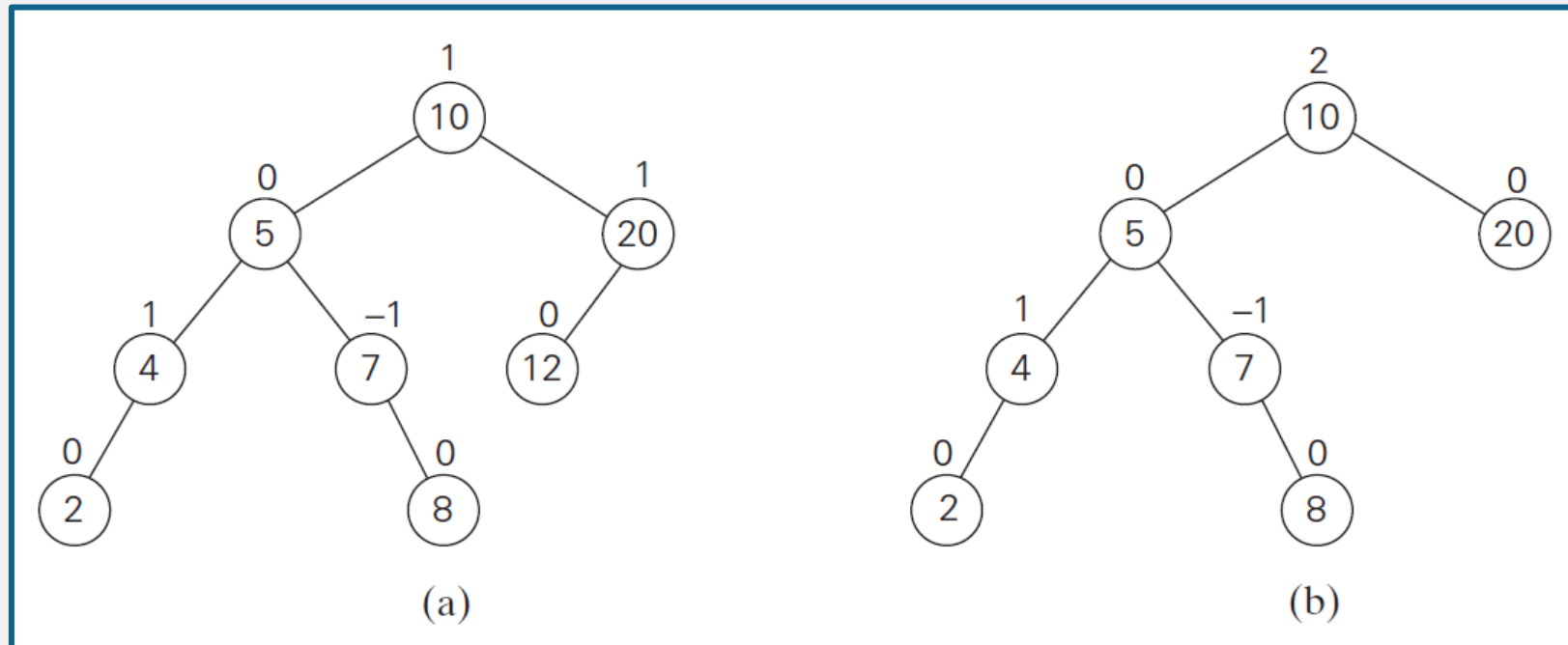
Attractiveness of *binary search tree* is marred by the bad (linear) worst-case efficiency. Two ideas to overcome it are:

- to rebalance binary search tree when a new insertion makes the tree “too unbalanced”
 - *AVL trees*
 - *red-black trees*
- to allow more than one key per node of a search tree
 - *2-3 trees*
 - *2-3-4 trees*
 - *B-trees*

Transform & Conquer

Instance simplification - Balanced Trees: AVL Tree

An *AVL tree* is a binary search tree in which, for every node, the difference between the heights of its left and right subtrees, called the *balance factor*, is at most 1 (with the height of an empty tree defined as -1)

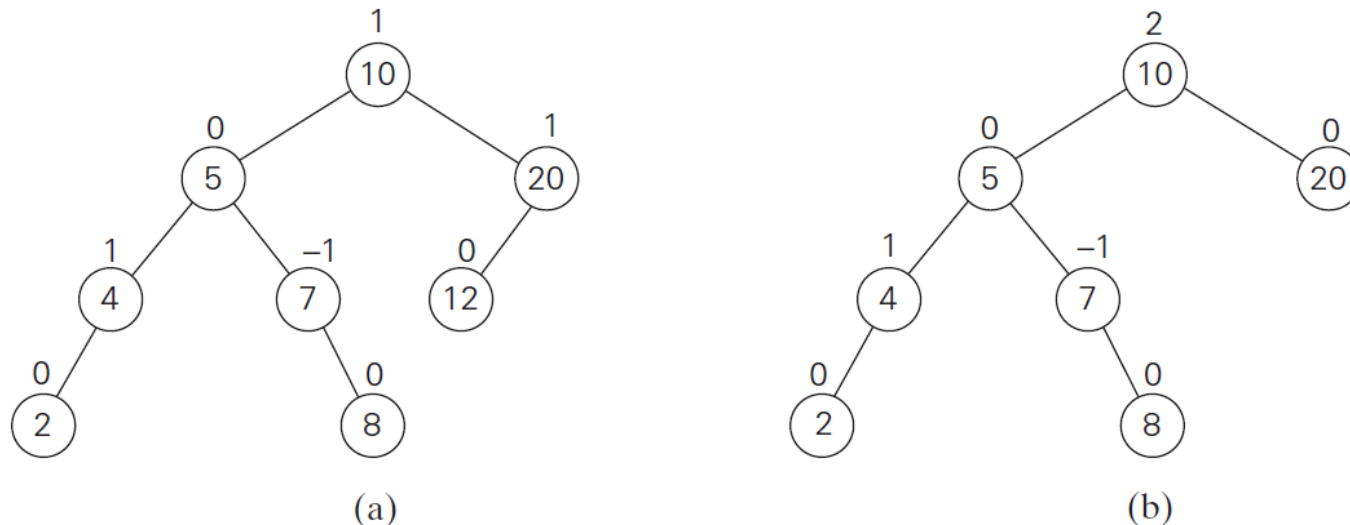


(a) AVL tree. (b) Binary search tree that is not an AVL tree.

Transform & Conquer

Instance simplification - Balanced Trees: AVL Tree

- The **height** of a tree is the number of edges between the root and the lowest leaf. The height of an empty tree is -1.
- The **balance factor** of the node of a tree is the difference between the heights of its left and right subtrees.
- An **AVL tree** is a binary search tree in which the absolute value of the balance factor of any node is at most 1

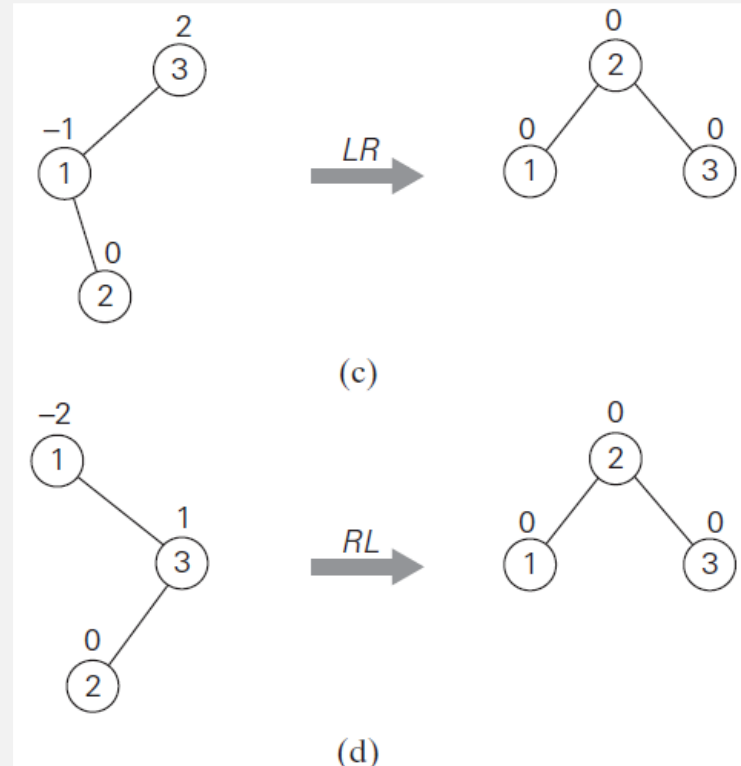
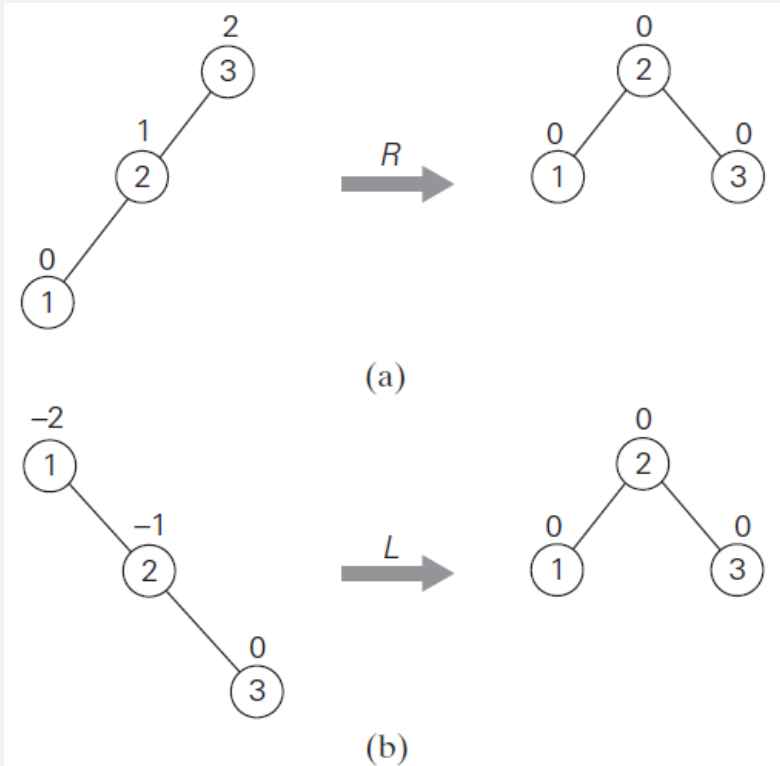


(a) AVL tree.
(b) Binary search tree (not an AVL tree).

Transform & Conquer

Instance simplification - Balanced Trees: AVL Tree - Rotations

If a key insertion violates the balance requirement at some node, the subtree rooted at that node is transformed via one of the four *rotations*. (The rotation is always performed for a subtree rooted at an “unbalanced” node closest to the new leaf.)

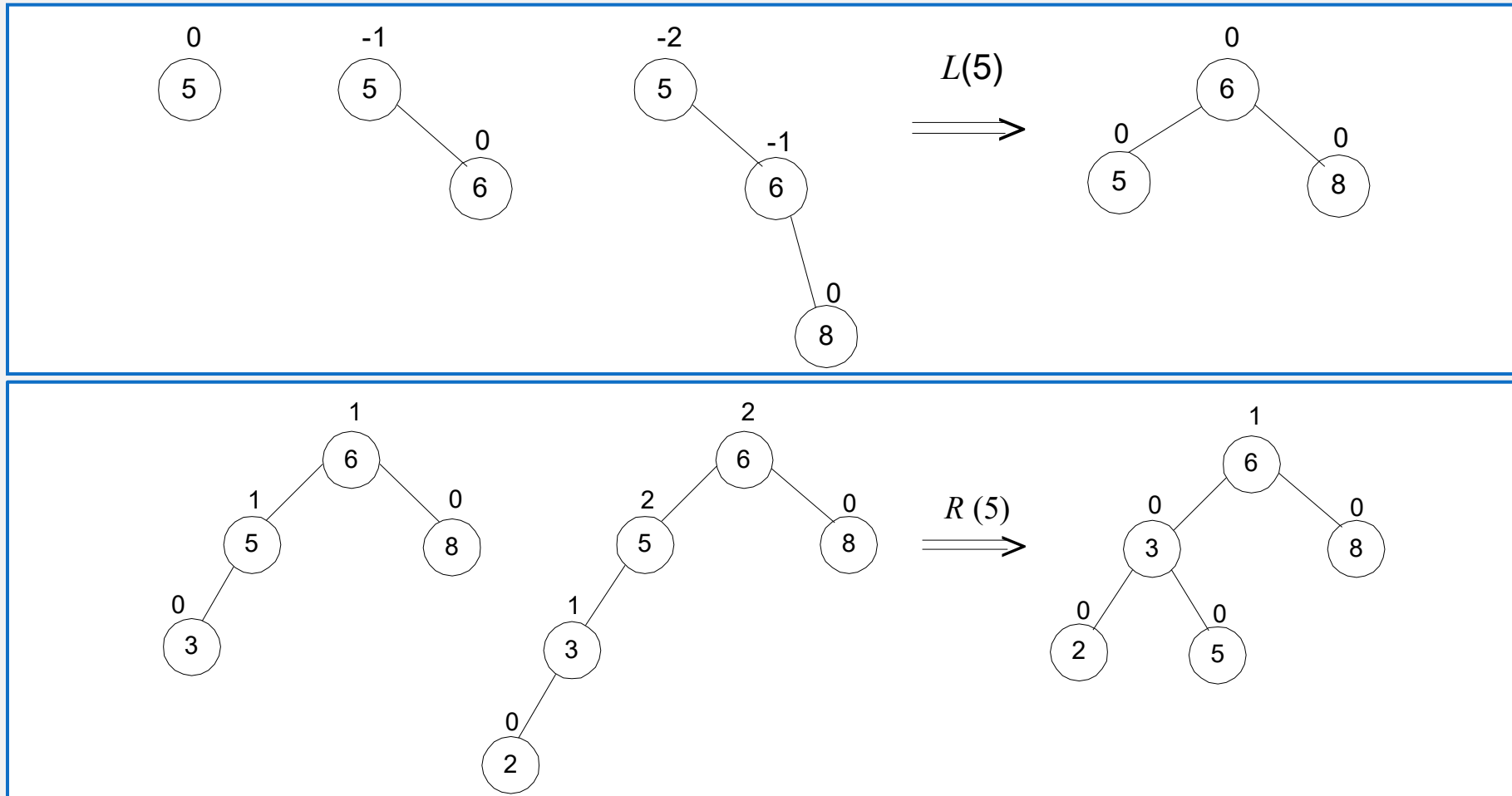


- (a) Single *R*-rotation.
- (b) Single *L*-rotation.
- (c) Double *LR*-rotation.
- (d) Double *RL*-rotation.

Transform & Conquer

Instance simplification - Balanced Trees: AVL Tree - Rotations

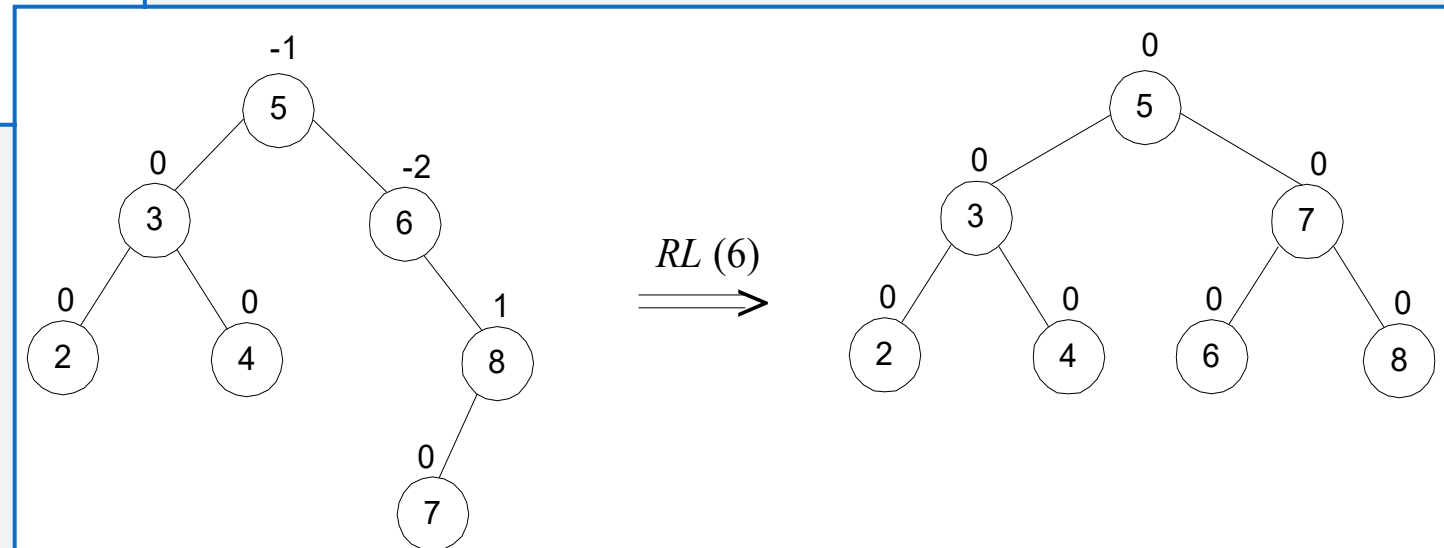
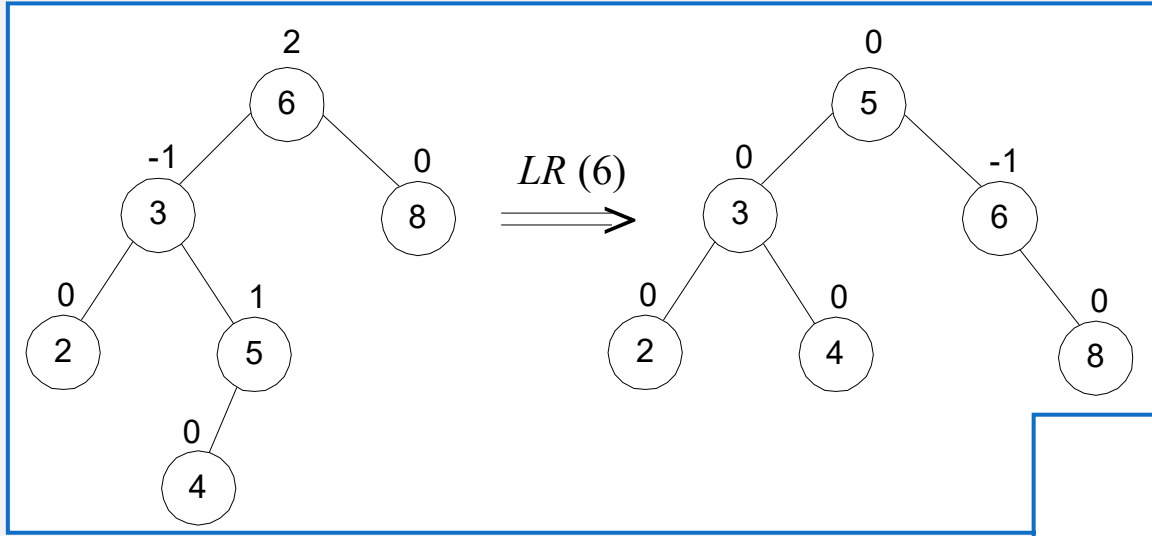
Construct an AVL tree for the list 5, 6, 8, 3, 2, 4, 7



Transform & Conquer

Instance simplification - Balanced Trees: AVL Tree - Rotations

Construct an AVL tree for the list 5, 6, 8, 3, 2, 4, 7



Transform & Conquer

Instance simplification - Balanced Trees: AVL Tree

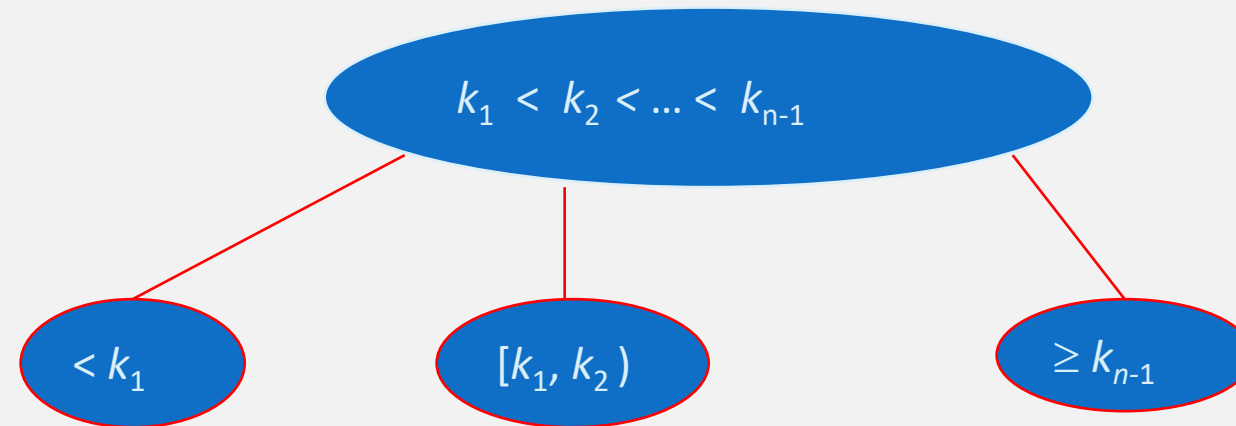
- $h \leq 1.4404 \log_2 (n + 2) - 1.3277$
- average height: $1.01 \log_2 n + 0.1$ for large n (found empirically)
- Search and insertion are $O(\log n)$
- Deletion is more complicated but is also $O(\log n)$
- Disadvantages:
 - frequent rotations
 - complexity
- A similar idea: red-black trees (height of subtrees is allowed to differ by up to a factor of 2)

Transform & Conquer

Representation change - Multiway Search Trees

Definition A *multiway search tree* is a search tree that allows more than one key in the same node of the tree.

Definition A node of a search tree is called an *n-node* if it contains $n-1$ ordered keys (which divide the entire key range into n intervals pointed to by the node's n links to its children)



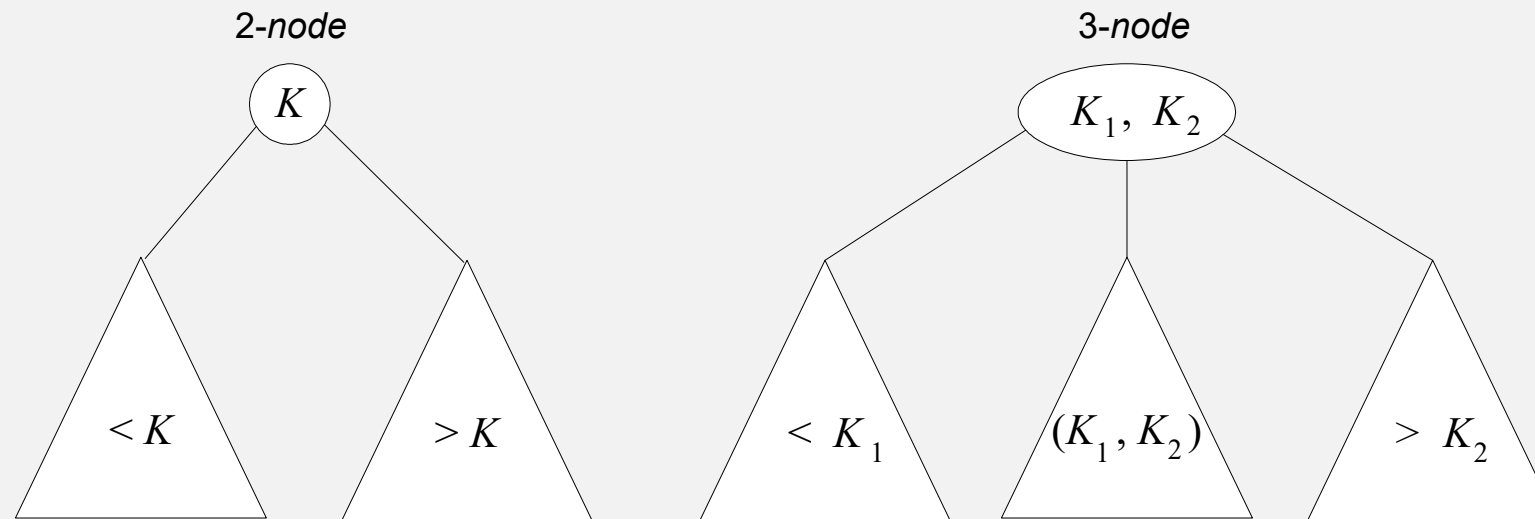
Note: Every node in a classical binary search tree is a 2-node

Transform & Conquer

Representation change - 2-3 Tree

A 2-3 tree is a search tree that

- may have 2-nodes and 3-nodes
- height-balanced (all leaves are on the same level)

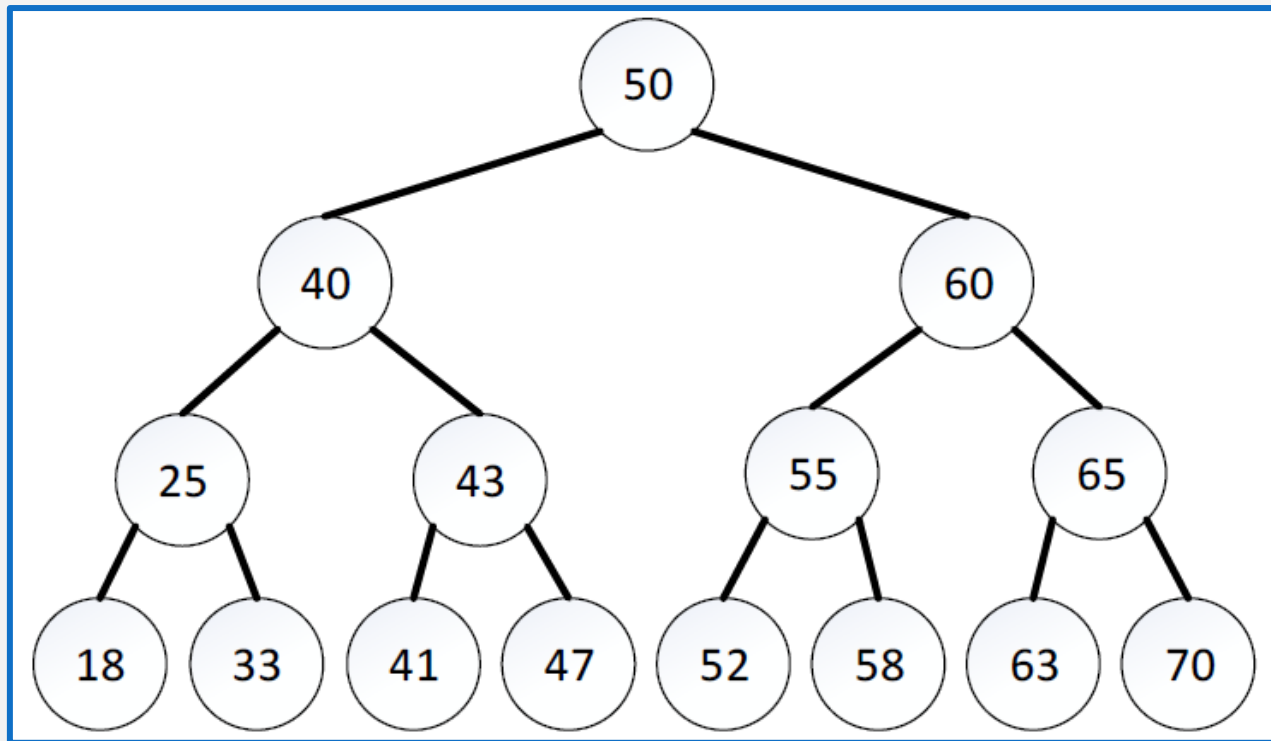


A 2-3 tree is constructed by successive insertions of keys given, with a new key always inserted into a leaf of the tree. If the leaf is a 3-node, it's split into two with the middle key promoted to the parent

Transform & Conquer

Representation change - 2-3 Tree

Max height when all nodes are 2-nodes



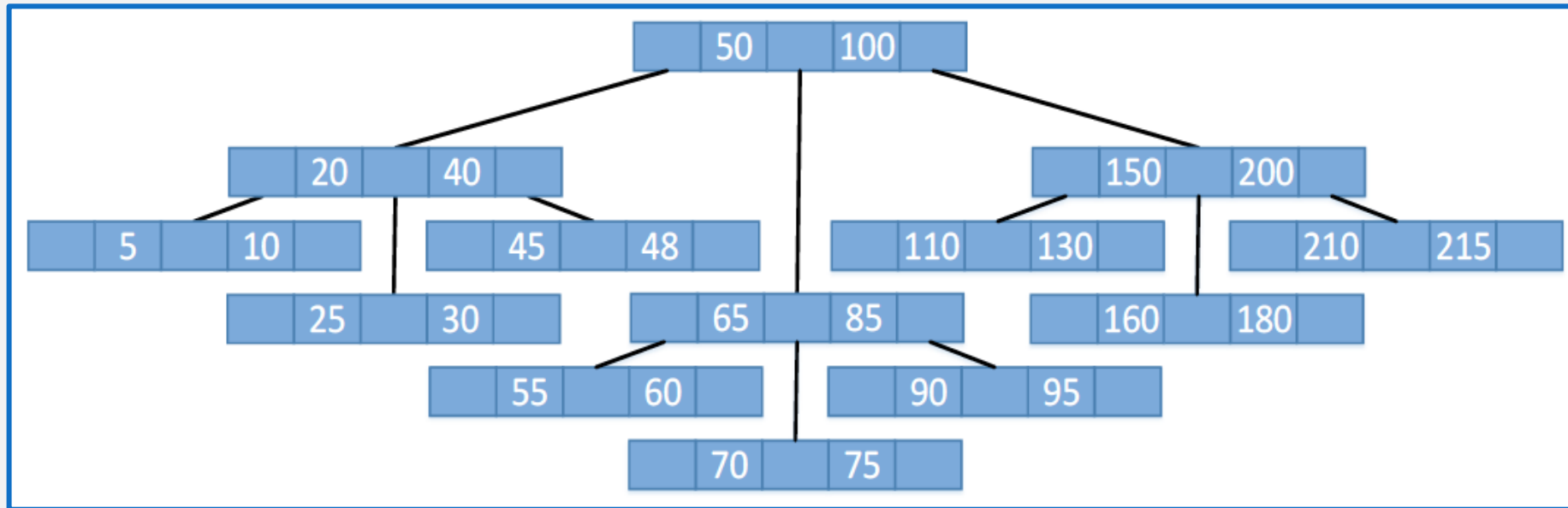
$$\begin{aligned} n &= \sum_{i=0}^{\text{height}} 2^i \\ &= \frac{(2^{\text{height}+1} - 1)}{2 - 1} \\ &= (2^{\text{height}+1} - 1) \end{aligned}$$

So height = $\log_2(n+1)-1$

Transform & Conquer

Representation change - 2-3 Tree

Min height when all nodes are 3-nodes

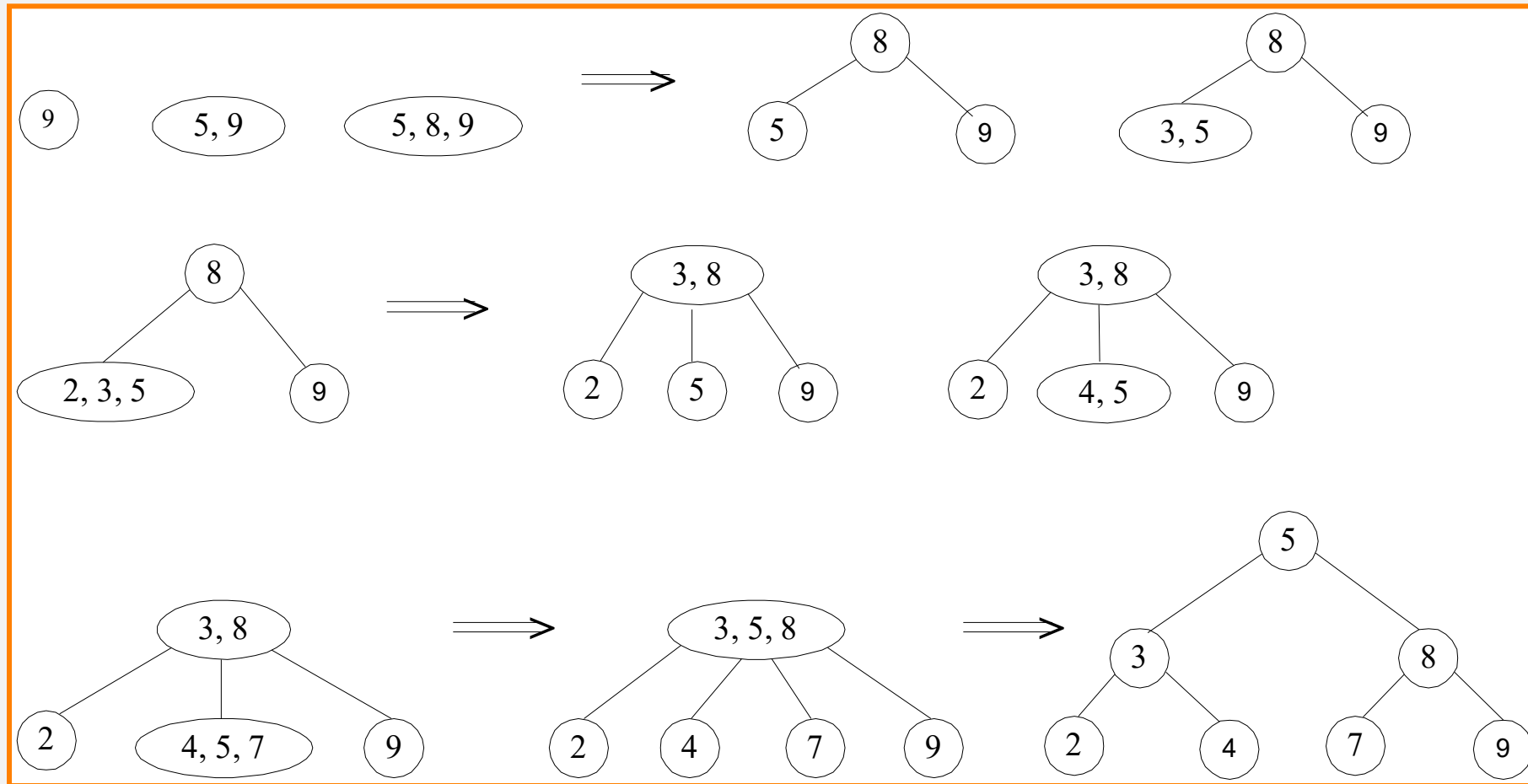


$$n = 2 \cdot \sum_{i=0}^{\text{height}} 3^i = 2 \cdot \frac{(3^{\text{height}+1} - 1)}{3 - 1} = (3^{\text{height}+1} - 1) = \log_3(n+1) - 1$$

Transform & Conquer

Representation change - 2-3 Tree

Construct a 2-3 tree the list 9, 5, 8, 3, 2, 4, 7



Analysis of 2-3 trees

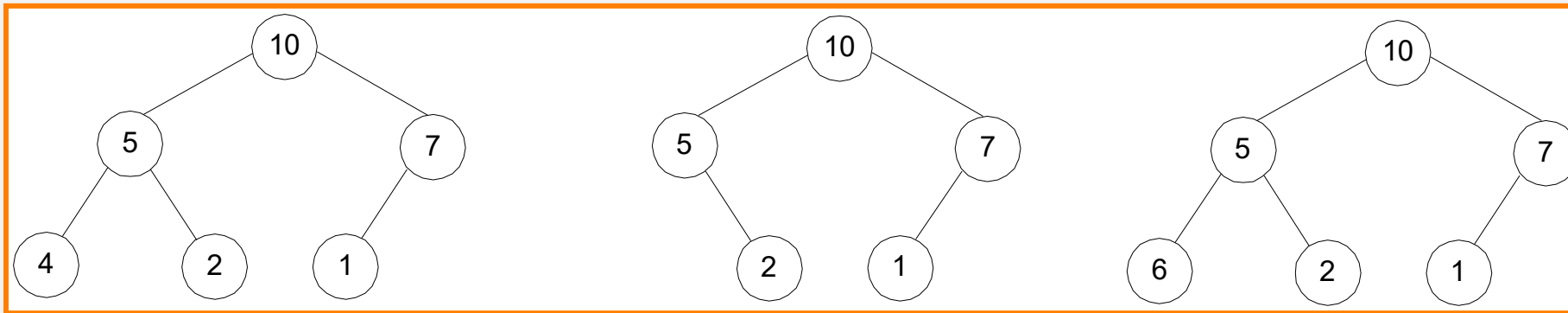
- $\log_3 (n + 1) - 1 \leq h \leq \log_2 (n + 1) - 1$
- Search, insertion, and deletion are in $\Theta(\log n)$
- The idea of 2-3 tree can be generalized by allowing more keys per node
 - 2-3-4 trees
 - B-trees
- However, 2-3 tree implementation is complicated. These trees can instead be implemented as red-black BSTs

Transform & Conquer

Representation change - Heaps

A *heap* is a binary tree with keys at its nodes (one key per node) such that:

- It is essentially complete, i.e., all its levels are full except possibly the last level, where only some rightmost keys may be missing
- The key at each node is $\geq$ keys at its children

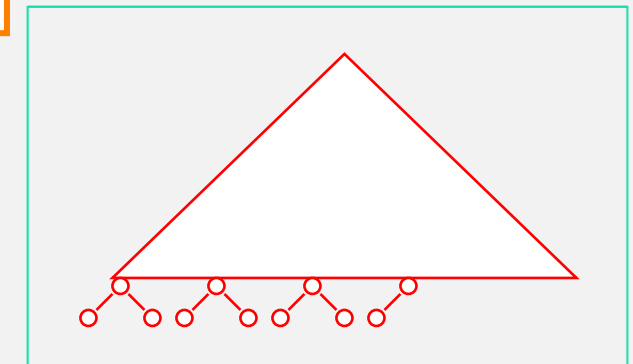


a heap

Not a heap

Not a heap

Note: Heap's elements are ordered top down (along any path down from its root), but they are not ordered left to right



Transform & Conquer

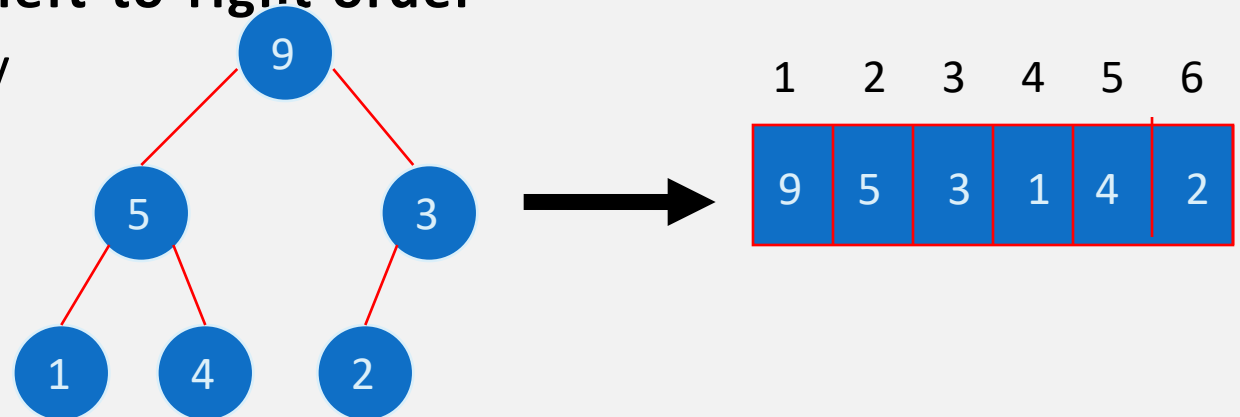
Representation change - Heaps

Some Important Properties of a Heap

- Given n , there exists a unique binary tree with n nodes that is essentially complete, with $h = \lfloor \log_2 n \rfloor$
- The root contains the largest key
- The subtree rooted at any node of a heap is also a heap

“Store heap’s elements in an array (whose elements indexed, for convenience, 1 to n) in top-down left-to-right order”

- A heap can be represented as an array
 - Left child of node j is at $2j$
 - Right child of node j is at $2j+1$
 - Parent of node j is at $\lfloor j/2 \rfloor$
 - Parental nodes are represented in the first $\lfloor n/2 \rfloor$ locations



Transform & Conquer

Representation change - Heaps

Heap construction

Step 0:

Initialize the structure with keys in the order given

Step 1:

Starting with the last (rightmost) parental node, fix the heap rooted at it, if it doesn't satisfy the heap condition: keep exchanging it with its largest child until the heap condition holds

Step 2:

Repeat Step 1 for the preceding parental node

ALGORITHM *HeapBottomUp*($H[1..n]$)

//Constructs a heap from elements of a given array

// by the bottom-up algorithm

//Input: An array $H[1..n]$ of orderable items

//Output: A heap $H[1..n]$

for $i \leftarrow n/2$ **downto** 1 **do**

$k \leftarrow i$; $v \leftarrow H[k]$

$heap \leftarrow \text{false}$

while not $heap$ **and** $2 * k \leq n$ **do**

$j \leftarrow 2 * k$

if $j < n$ //there are two children

if $H[j] < H[j + 1]$

$j \leftarrow j + 1$

if $v \geq H[j]$

$heap \leftarrow \text{true}$

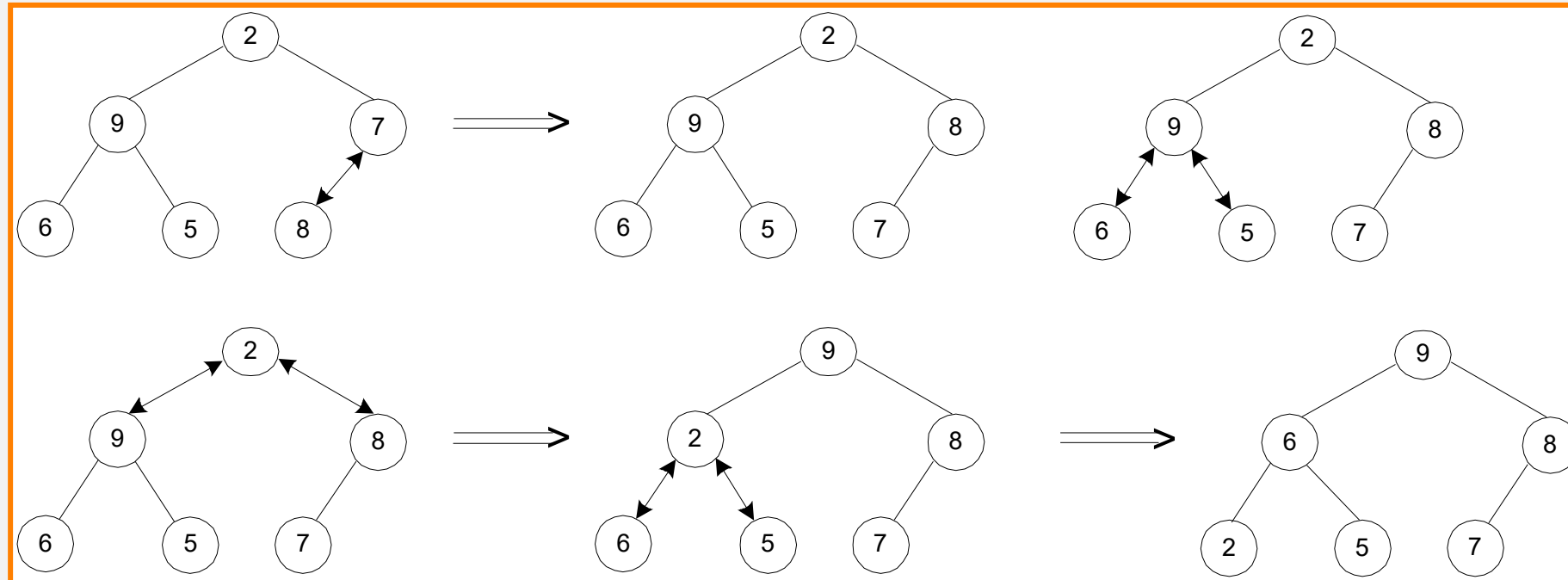
else $H[k] \leftarrow H[j]$; $k \leftarrow j$

$H[k] \leftarrow v$

Transform & Conquer

Representation change - Heaps

Heap construction



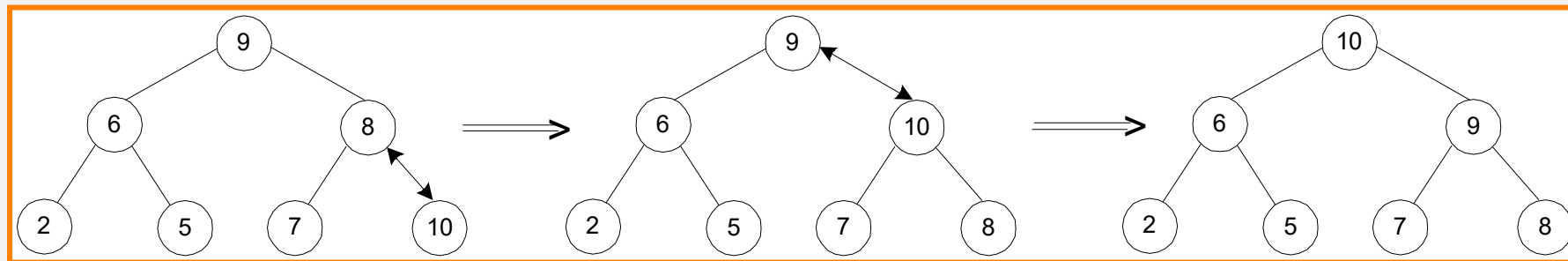
Bottom-up construction of a heap for the list 2, 9, 7, 6, 5, 8. The double headed arrows show key comparisons verifying the parental dominance.

Transform & Conquer

Representation change - Heaps

Inserting in a Heap

- Insert the new element at last position in heap.
- Compare it with its parent and, if it violates heap condition, exchange them
- Continue comparing the new element with nodes up the tree until the heap condition is satisfied



Insert key 10

- Efficiency: $O(\log n)$

Transform & Conquer

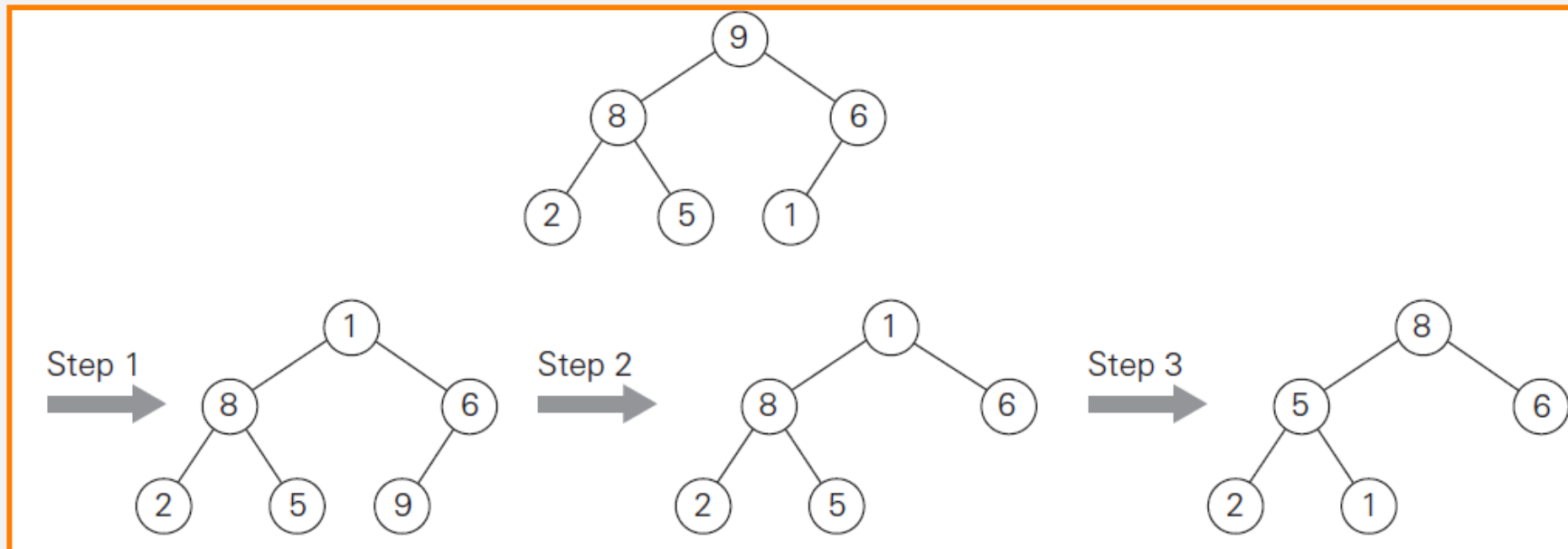
Representation change - Heaps

Maximum Key Deletion from a heap

Step 1 Exchange the root's key with the last key K of the heap.

Step 2 Decrease the heap's size by 1.

Step 3 "Heapify" the smaller tree by sifting K down the tree. Verify the parental dominance for K : if it holds, we are done; if not, swap K with the larger of its children and repeat this operation until the parental dominance condition holds for K in its new position.



Efficiency: $O(\log n)$

Transform & Conquer

Representation change - Heapsort

Sorting a list using Heapsort

Stage 1: Construct a heap for a given list of n keys $\Theta(n)$

Stage 2: Repeat operation of root removal $n-1$ times: $\Theta(n \log n)$

- Exchange keys in the root and in the last (rightmost) leaf
- Decrease heap size by 1
- If necessary, swap new root with larger child until the heap condition holds

Transform & Conquer

Representation change - Heapsort

Sort the list 2, 9, 7, 6, 5, 8 by heapsort

Stage 1 (heap construction)

2	9	<u>7</u>	6	5	8
2	<u>9</u>	8	6	5	7
<u>2</u>	9	8	6	5	7
9	<u>2</u>	8	6	5	7
9	6	8	2	5	7

Stage 2 (root/max removal)

<u>9</u>	6	8	2	5	7
7	6	8	2	5	9
<u>8</u>	6	7	2	5	9
5	6	7	2	8	9
<u>7</u>	6	5	2	8	9

2	6	5		7	8	9
<u>6</u>	2	5		7	8	9
5	2		6	7	8	9
<u>5</u>	2		6	7	8	9
2		5	6	7	8	9

Analysis of Heapsort

Stage 1: Build heap for a given list of n keys

worst-case

$$C(n) = \sum_{i=0}^{h-1} 2^{(h-i)} 2^i = 2(n - \log_2(n+1)) \in \Theta(n)$$

nodes at
level i

Stage 2: Repeat operation of root removal $n-1$ times (fix heap)

worst-case

$$C(n) = \sum_{i=1}^{n-1} 2\log_2 i \in \Theta(n \log n)$$

Both worst-case and average-case efficiency: $\Theta(n \log n)$

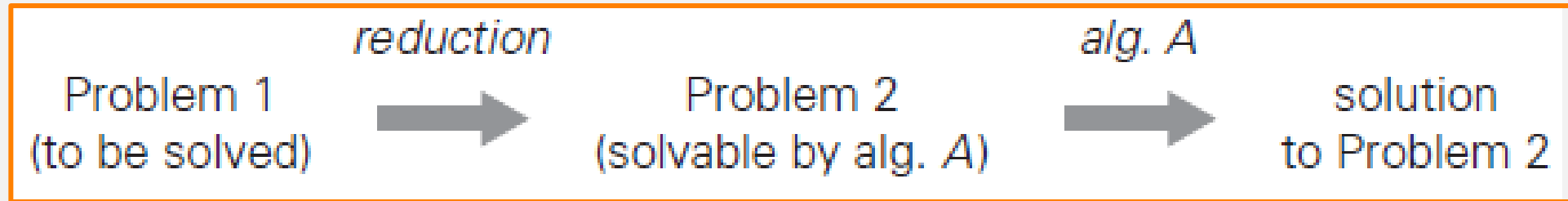
In-place: yes

Stability: no (e.g., 1 1)

Transform & Conquer

Problem reduction

- Definition: solve a problem by a transforming it into different problem for which an algorithm is already available.



- Examples:
 - Computing the Least Common Multiple
 - Counting number of paths of length n in a graph
 - Transforming a maximization problem to a minimization problem
 - Linear programming
 - Reduction to graph problems

Problem Reduction

This variation of transform-and-conquer solves a problem by transforming it into different problem for which an algorithm is already available.

To be of practical value, the combined time of the transformation and solving the other problem should be smaller than solving the problem as given by another method.

Transform & Conquer

Problem reduction -The Least Common Multiple

- The least common multiple of two positive integers m and n , denoted $\text{lcm}(m,n)$, is the smallest integer that is divisible by both m and n .
- Solution 1: compute the product of all the common prime factors of m and n , all the prime factors of m that are not in n , and all the prime factors of n that are not in m .

$$24 = 2 \cdot 2 \cdot 2 \cdot 3,$$

$$60 = 2 \cdot 2 \cdot 3 \cdot 5,$$

$$\text{lcm}(24, 60) = (2 \cdot 2 \cdot 3) \cdot 2 \cdot 5 = 120$$

- Solution 2:

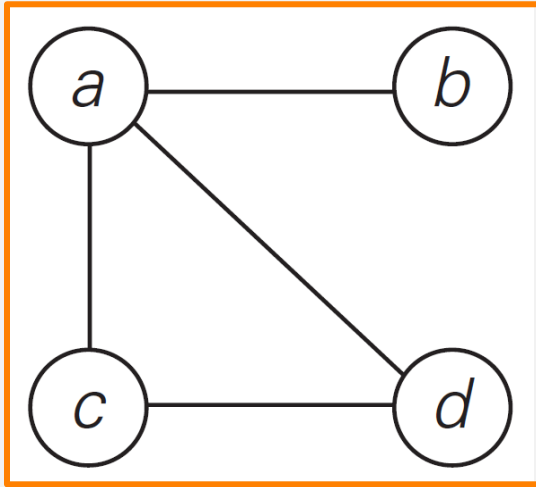
Theorem: $m \cdot n = \text{gcd}(m,n) \times \text{lcm}(m,n)$ and so $\text{lcm}(m, n) = \frac{m \cdot n}{\text{gcd}(m,n)}$

- Algorithm:
 - Calculate $\text{gcd}(m,n)$ first using Euclidian Algorithm
 - Apply formula above

Transform & Conquer

Problem reduction –Counting paths in a graph

Problem: given a graph, how many paths of size n does it have?



1. Represent graph by adjacency matrix A

$$A = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

paths of size 2

$$A^2 = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 3 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 2 & 1 \\ 1 & 1 & 1 & 2 \end{bmatrix} \end{matrix}$$

Theorem:

The number of paths of length n from vertex i to j is $A^n[i,j]$

take the power of A n times - answers are in matrix A^n

Transform & Conquer

Problem reduction –Linear Programming

- Many problems of optimal decision making can be reduced to an instance of the **linear programming** problem.
- Linear programming** problem: problem of optimizing a linear function of several variables subject to constraints in the form of linear equations and linear inequalities.

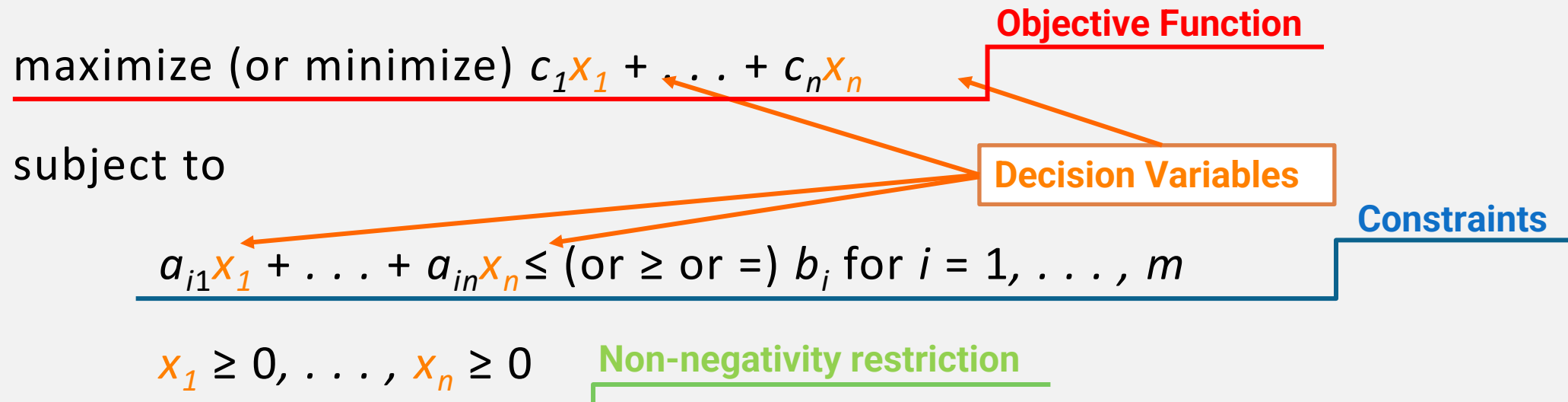
maximize (or minimize) $c_1x_1 + \dots + c_nx_n$ Objective Function

subject to

$a_{i1}x_1 + \dots + a_{in}x_n \leq (\text{or } \geq \text{ or } =) b_i \text{ for } i = 1, \dots, m$ Constraints

$x_1 \geq 0, \dots, x_n \geq 0$ Non-negativity restriction

Decision Variables



Transform & Conquer

Problem reduction –Linear Programming

EXAMPLE 1

Consider that needs to invest \$100 million. This sum has to be split between three types of investments: A, B, and C.

We expect an annual return of 10%, 7%, and 3% for A, B, and C investments, respectively. Since investment A are more risky than B, the investment rules require the amount invested in A to be no more than one-third of the moneys invested in B. In addition, at least 25% of the total amount invested in A and B must be invested in C. How should we invest the money to maximize the return?

Decision variables

- x: amount invested in A
- y: amount invested in B
- z: amount invested in C

Linear program

maximize $0.10x + 0.07y + 0.03z$
subject to

$$x + y + z = 100$$

$$x \leq \frac{1}{3}y$$

$$z \geq 0.25(x + y)$$

$$x \geq 0, y \geq 0, z \geq 0$$

Optimal Solution

X=20; y=60; z=20

Objective function: 6.8

Transform & Conquer

Problem reduction –Linear Programming

EXAMPLE 2: Reducing the Knapsack problem to a linear problem

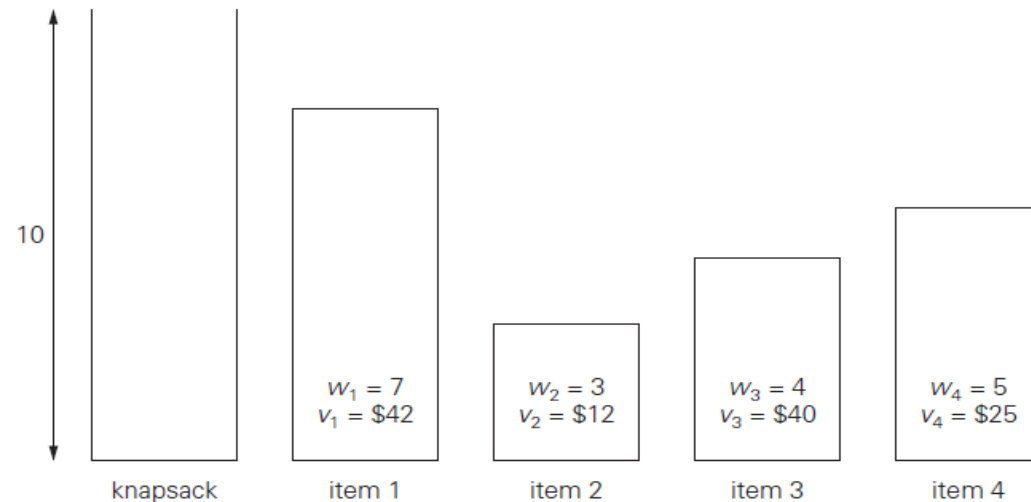
Given a knapsack of capacity W and n items of weights $w_1, \dots, w_n$ and values $v_1, \dots, v_n$, find the most valuable subset of the items that fits into the knapsack.

$$\begin{aligned} &\text{maximize} && \sum_{j=1}^n v_j x_j \\ &\text{subject to} && \sum_{j=1}^n w_j x_j \leq W \\ &&& x_j \in \{0, 1\} \quad \text{for } j = 1, \dots, n. \end{aligned}$$

Transform & Conquer

Problem reduction –Linear Programming

EXAMPLE 2: Reducing the Knapsack problem to a linear problem



Linear program
maximize $42x_1 + 12x_2 + 40x_3 + 25x_4$
subject to
 $7x_1 + 3x_2 + 4x_3 + 5x_4 \leq 10$
 $x_1, x_2, x_3, x_4 \in \{0,1\}$

Optimal Solution

$$x_1 = 0$$

$$x_2 = 0$$

$$x_3 = 1$$

$$x_4 = 1$$

Objective function: 65

Transform & Conquer

Problem reduction –Linear Programming

EXAMPLE 3:Chocolate manufacturer

Consider a chocolate manufacturing company that produces only two types of chocolate – A and B. Both the chocolates require Milk and Choco only. To manufacture each unit of A and B, the following quantities are required:

- Each unit of A requires 1 unit of Milk and 3 units of Choco
- Each unit of B requires 1 unit of Milk and 2 units of Choco

The company kitchen has a total of 5 units of Milk and 12 units of Choco. On each sale, the company makes a profit of:

- 6 sar per unit A sold
- 5 sar per unit B sold

Now, the company wishes to maximize its profit. How many units of A and B should it produce respectively?

Decision variables

The total number of units produced by A be = X

The total number of units produced by B be = Y

Linear program

maximize $6X + 5Y$

subject to $X + Y \leq 5$

$3X + 2Y \leq 12$

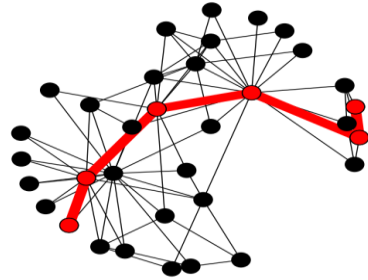
$X \geq 0$ & $Y \geq 0$

Transform & Conquer

Problem reduction – Reduction to Graph Problems

Shortest Path Problem

- Routing internet traffic
- Travelling salesman problem



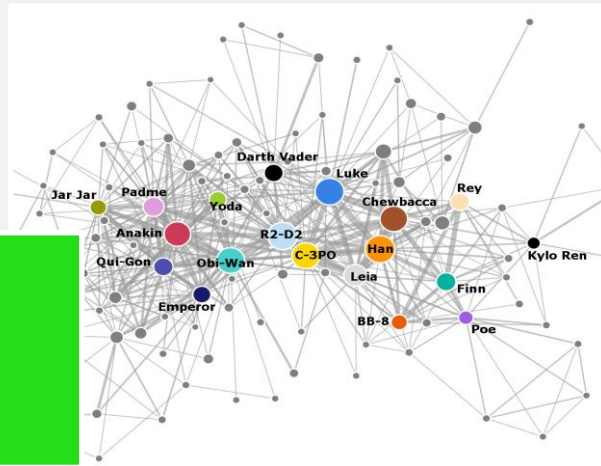
Longest path | Topological sorting

- Scheduling and planning



Social Networks

- Find influential people
- Clique for ads



Matching

- Organ donation
- Matching users to servers

